

APIs & Communication Contracts

Phase 1 · Fundamentals · Estimated time: 1.5 hours

1. Concept From First Principles

English:

An API (Application Programming Interface) is a contract: it defines exactly how one piece of software can ask another for something, without either side needing to know the other's internal implementation. HLD asks you to think about that contract at the scale of services talking to services.

Hindi:

एक API (Application Programming Interface) एक contract होता है: यह बिल्कुल तय करता है कि सॉफ्टवेयर का एक हिस्सा दूसरे से कुछ कैसे मांग सकता है, बिना यह जाने कि दूसरे के अंदर implementation कैसी है। HLD आपसे कहता है कि इस contract के बारे में services-to-services के स्तर पर सोचें।

Important Words:

- **Contract (figurative)** – अनुबंध / तय समझौता

Synchronous vs. Asynchronous communication

English:

Synchronous communication means the caller sends a request and blocks, waiting for a response before continuing. Asynchronous communication means the caller sends a message and moves on immediately, without waiting for the work to finish.

Hindi:

Synchronous communication का मतलब है कि caller एक request भेजता है और आगे बढ़ने से पहले response का इंतज़ार करता है (block होता है)। Asynchronous communication का मतलब है कि caller एक message भेजकर तुरंत आगे बढ़ जाता है, काम पूरा होने का इंतज़ार किए बिना।

Important Words:

- **Block (verb)** – रुक जाना
- **Caller** – कॉलर / मांगने वाला

Backend engineer analogy

English:

Dispatching a Laravel Job onto a queue instead of running it inline in the controller is exactly the sync-vs-async decision. Sending a welcome email synchronously during signup makes the user wait for an SMTP round-trip they don't care about -- queueing it is the async, correct choice.

Hindi:

Laravel Job को controller में inline चलाने की बजाय queue पर dispatch करना बिल्कुल वही sync-vs-async decision है। Signup के समय welcome email synchronously भेजना user को एक SMTP round-trip के लिए रोक देता है जिसकी उसे परवाह भी नहीं -- इसे queue पर डालना ही सही, async choice है।

REST vs. gRPC

English:

REST represents resources as URLs and uses standard HTTP verbs and JSON payloads. It's human-readable, universally supported, and the default choice for public-facing and browser-facing APIs.

Hindi:

REST resources को URLs के रूप में दिखाता है और standard HTTP verbs और JSON payloads use करता है। यह इंसानों के पढ़ने लायक है, हर जगह supported है, और public-facing व browser-facing APIs के लिए default choice है।

Important Words:

- **Payload** – पेलोड / भेजा जाने वाला डेटा

English:

gRPC is a binary RPC framework built on HTTP/2, using Protocol Buffers instead of JSON. It's significantly faster and smaller on the wire than REST/JSON, supports streaming natively, and enforces a strict, typed contract between services.

Hindi:

gRPC एक binary RPC framework है जो HTTP/2 पर बना है, और JSON की जगह Protocol Buffers use करता है। यह REST/JSON से काफी तेज़ और wire पर छोटा होता है, streaming को natively support करता है, और services के बीच एक strict, typed contract को enforce करता है।

Important Words:

- **Binary** – बाइनरी
- **Streaming** – स्ट्रीमिंग
- **Typed contract** – टाइप्ड कॉन्ट्रैक्ट

English:

One more foundational idea: whenever you make a call that might be retried, you need to think about idempotency -- can this operation be safely repeated without causing a duplicate side effect? A well-designed API contract makes this explicit, often via an idempotency key.

Hindi:

एक और ज़रूरी बात: जब भी कोई call ऐसी हो जो retry हो सकती है, आपको idempotency के बारे में सोचना चाहिए -- क्या इस operation को दोबारा करना सुरक्षित है, बिना कोई duplicate side effect पैदा किए? एक अच्छी तरह design किया गया API contract इसे साफ-साफ बताता है, अक्सर एक idempotency key के ज़रिए।

2. Real-World Example / Case Study

English:

Razorpay's public payment API is REST -- merchants integrate over standard HTTPS/JSON because it needs to be usable by every programming language and skill level. Internally, high-throughput microservices often use gRPC between themselves.

Hindi:

Razorpay का public payment API REST है -- merchants standard HTTPS/JSON पर integrate करते हैं क्योंकि इसे हर programming language और हर skill level के लोग use कर सकें। अंदरूनी तौर पर, high-throughput microservices अक्सर आपस में gRPC use करती हैं।

Important Words:

- **Integrate** – इंटीग्रेट करना / जोड़ना

English:

Swiggy's order flow illustrates sync vs. async well: the checkout API call is synchronous -- the user is waiting and needs a real answer. But triggering the kitchen printer, sending push notifications, and updating analytics are all fired asynchronously via a queue.

Hindi:

Swiggy का order flow sync vs. async को अच्छे से दिखाता है: checkout API call synchronous है -- user इंतज़ार कर रहा है और उसे असली जवाब चाहिए। लेकिन kitchen printer चलाना, push notification भेजना, और analytics update करना -- यह सब queue के ज़रिए asynchronously किया जाता है।

Important Words:

- **Trigger (verb)** – शुरू करना / ट्रिगर करना

English:

A third example: PhonePe's merchant settlement API exposes both a synchronous 'check status' endpoint and an asynchronous webhook. Offering both is a common, deliberate pattern -- sync for on-demand checks, async for timely notification.

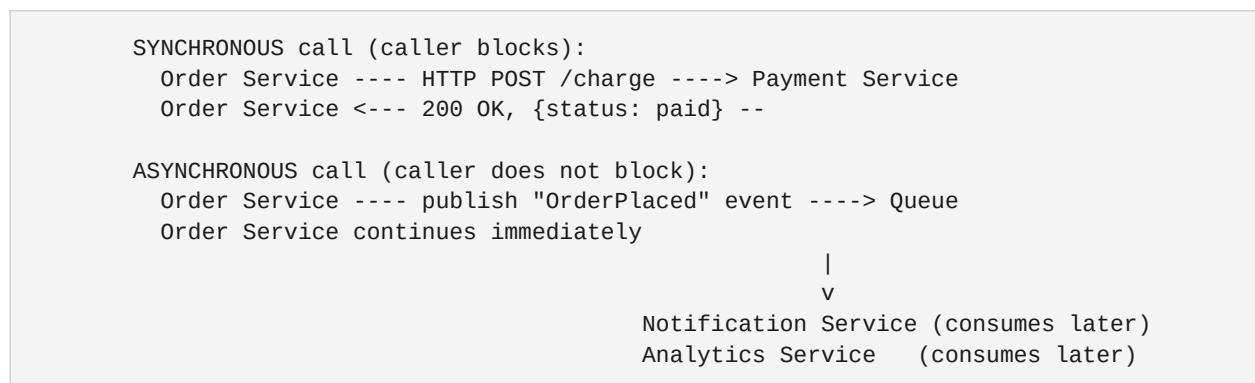
Hindi:

एक तीसरा उदाहरण: PhonePe का merchant settlement API एक synchronous 'check status' endpoint और एक asynchronous webhook, दोनों देता है। दोनों देना एक आम और जानबूझकर किया गया pattern है -- on-demand check के लिए sync, समय पर सूचना के लिए async।

Important Words:

- **On-demand** – मांग पर / जब चाहे तब

3. Diagram (English labels kept as-is)



4. Trade-offs Table (Reference Table – English)

Aspect	REST (JSON over HTTP)	gRPC (Protobuf over HTTP/2)
Readability	Human-readable, browser-debuggable	Binary, needs tooling

Aspect	REST (JSON over HTTP)	gRPC (Protobuf over HTTP/2)
Performance	Slower, larger payloads	Faster, smaller, native streaming
Typing	Loose unless enforced separately	Strict, code-generated
Best for	Public APIs, browser clients	Internal service-to-service

5. Common Interview Questions

Q1: "When would you choose gRPC over REST?"

English:

Model approach: State that gRPC shines for internal, high-volume, latency-sensitive service-to-service calls with a stable contract, while REST remains better for public APIs or anything a browser needs to call directly.

Hindi:

Model approach: बताएं कि gRPC internal, high-volume, latency-sensitive service-to-service calls के लिए बेहतरीन है जहाँ contract stable हो, जबकि REST public APIs या browser से सीधे call होने वाली चीज़ों के लिए बेहतर रहता है।

Q2: "How do you decide if a call should be synchronous or asynchronous?"

English:

Model approach: Ask 'does the caller need the result to proceed, and does the user expect an immediate answer?' If yes to both, keep it synchronous. If the work can happen in the background, make it asynchronous via a queue.

Hindi:

Model approach: पूछें 'क्या caller को आगे बढ़ने के लिए result चाहिए, और क्या user तुरंत जवाब की उम्मीद करता है?' अगर दोनों का जवाब हाँ है, तो synchronous रखें। अगर काम background में हो सकता है, तो उसे queue के ज़रिए asynchronous बनाएं।

Q3: "How would you design an API so a duplicate 'place order' request doesn't create two orders?"

English:

Model approach: Require the client to send a unique idempotency key; the server checks whether it has already processed that key, and if so, returns the original response instead of creating a duplicate.

Hindi:

Model approach: client से एक unique idempotency key भेजने के लिए कहें; server check करे कि क्या वह key पहले से process हो चुकी है, अगर हाँ तो नया order बनाने की बजाय पुराना ही response वापस दे।

6. Practice Exercises

English:

- For an e-commerce checkout, list 3 calls that must be synchronous and 3 that should be asynchronous.

- Explain the core difference between REST and gRPC to a non-technical friend using an analogy.
- Sketch an API contract for 'place an order' as if you were designing it today.

Hindi:

- एक e-commerce checkout के लिए, 3 calls बताएं जो synchronous होनी चाहिए और 3 जो asynchronous होनी चाहिए।
- REST और gRPC का मुख्य फर्क किसी non-technical दोस्त को एक analogy से समझाएं।
- 'place an order' के लिए एक API contract बनाएं जैसे आप आज इसे design कर रहे हों।

7. Curated YouTube Videos (Reference – English)

Language	Video & Channel	Why it helps
English	"REST vs gRPC in Microservices Which One Should You Use?"	Practical, decision-focused.
English	"gRPC vs REST API – Speed, Streaming & Scalability Explained"	Focuses on performance trade-offs.
Hindi	Same REST vs gRPC video (widely referenced, accessible to Hindi-English learners)	Common reference point.
Hindi	"Complete System Design Roadmap 2025 HLD & LLD" — CodeHelp	Covers API design choices in Hindi.

8. Key Takeaways

English:

- APIs are contracts between software components, independent of internal implementation.
- Synchronous calls block the caller for a result; asynchronous calls let the caller move on immediately.
- Choose sync when the caller genuinely needs the result now; choose async otherwise.
- REST is the default for public and browser-facing APIs.
- gRPC is preferred for internal, high-throughput, latency-sensitive service calls.
- Real production systems mix both: REST at the edge, gRPC or async messaging internally.

Hindi:

- APIs सॉफ्टवेयर के हिस्सों के बीच contracts होते हैं, internal implementation से अलग।
- Synchronous calls caller को result के लिए रोक देती हैं; asynchronous calls caller को तुरंत आगे जाने देती हैं।
- जब caller को अभी result चाहिए हो तभी sync चुनें; बाकी हालात में async चुनें।
- Public और browser-facing APIs के लिए REST डिफॉल्ट है।
- Internal, high-throughput, latency-sensitive service calls के लिए gRPC पसंद किया जाता है।
- असली production systems दोनों मिलाकर use करते हैं: बाहर REST, अंदर gRPC या async messaging।

General Words Glossary

A consolidated list of the important English words used throughout this chapter, with Hindi meanings and simple explanations — useful for revising vocabulary after finishing the chapter.

English Word	Hindi Meaning	Simple Explanation
Binary	बाइनरी	Data in raw 0s and 1s form, not text
Block (verb)	रुक जाना	To pause and wait until something completes
Caller	कॉलर / मांगने वाला	The part of the system making the request
Contract (figurative)	अनुबंध / तय समझौता	An agreed way two parts will interact
Integrate	इंटीग्रेट करना / जोड़ना	To connect and make systems work together
On-demand	मांग पर / जब चाहे तब	Available whenever requested
Payload	पेलोड / भेजा जाने वाला डेटा	The actual data being transmitted
Streaming	स्ट्रीमिंग	Sending data continuously rather than all at once
Trigger (verb)	शुरू करना / ट्रिगर करना	To cause something to start
Typed contract	टाइप्ड कॉन्ट्रैक्ट	A contract where data shapes are strictly defined